



Name:	Lyka Sophia S. Jardeleza	Date:	06/27/2025
Program:	BSCpE	Section:	A361

A. Guided Exercises:

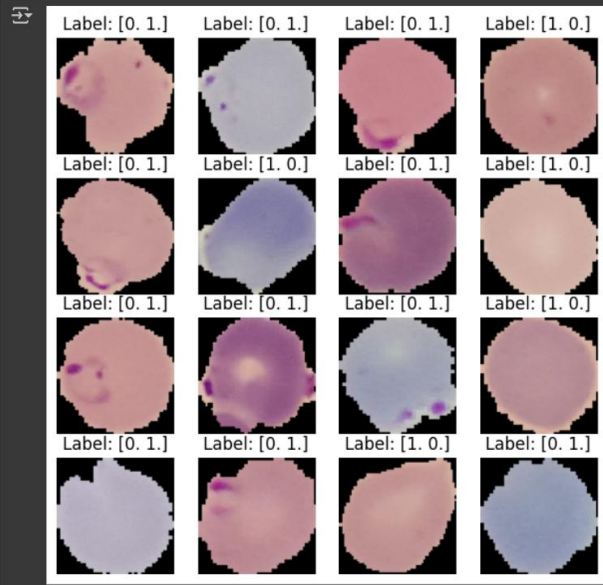
(Please include the following: 1. Photos of the codes and 2. Outputs after executing the Python code.)

1. Introduction to TensorFlow and Neural Networks

Photo/s of Python output	Photos of Python Codes in Colaboratory
a) Creating and initializing a neural network <pre>Model: "sequential" Layer (type) Output Shape Param # ----- dense (Dense) (None, 3) 9 dense_1 (Dense) (None, 1) 4 Total params: 13 (52.00 B) Trainable params: 13 (52.00 B) Non-trainable params: 0 (0.00 B) None</pre>	Laboratory Task #1 <pre>[4] import numpy as np import tensorflow as tf from tensorflow.keras import layers from tensorflow.keras.models import Sequential from tensorflow.keras.layers import Dense from tensorflow.keras.callbacks import Callback</pre>
b) Checking the output of an untrained model <pre>1/1 ----- 0s 358ms/step inputs: [[0. 0.]], output: -0.34245 1/1 ----- 0s 36ms/step inputs: [[0. 1.]], output: -0.43038 1/1 ----- 0s 36ms/step inputs: [[1. 0.]], output: -0.02091 1/1 ----- 0s 38ms/step inputs: [[1. 1.]], output: -0.09597</pre>	<pre>#a.) Creating and Initializing the Neural Network #train and test data X = tf.constant([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=tf.float32) Y = tf.constant([[0], [1], [1], [0]], dtype=tf.float32) #create the model #also initializes the model weights using random sampling from a normal distribution model = Sequential() model.add(tf.keras.Input(shape=(2,))) model.add(Dense(3, activation='sigmoid', kernel_initializer='glorot_normal')) model.add(Dense(1, kernel_initializer='glorot_normal')) #define the optimizer sgd = tf.keras.optimizers.SGD(learning_rate=0.02, momentum=0.9) #compile the model (which includes the loss function definition) model.compile(loss='mean_squared_error', optimizer=sgd) print(model.summary()) #b.) Check the Output of an Untrained Model #check the output of an untrained model for inputs in X: inputs = tf.expand_dims(inputs, 0) outputs = model.predict(inputs, steps=1) print('inputs: {}, output: {}'.format(inputs, round(outputs.item(), 5)))</pre>
c) Training the simple neural network <pre>Epoch: 1, Loss: 0.8803746104240417 Epoch: 101, Loss: 0.26114195585250854 Epoch: 201, Loss: 0.20309880375862122 Epoch: 301, Loss: 0.011447195895016193 Epoch: 401, Loss: 1.7056374872481683e-06 Epoch: 501, Loss: 2.1040857944853997e-10 Epoch: 601, Loss: 1.6608936448392342e-12 Epoch: 701, Loss: 7.691625114603085e-13 Epoch: 801, Loss: 9.29034627006331e-13 Epoch: 901, Loss: 6.608047442568932e-13 <keras.src.callbacks.history.History at 0x7914b5702c50></pre>	<pre>#c.) Training the Simple Neural Network #create a callback subclass to print only after N epochs class CustomLossUpdate(Callback): def on_epoch_end(self, epoch, logs={}): if epoch % 100 == 0: print('Epoch: {}, Loss: {}'.format(epoch + 1, logs.get('loss'))) #initialize the callback loss_update = CustomLossUpdate() #train the model model.fit(X, Y, verbose=0, steps_per_epoch=4, epochs=1000, callbacks=[loss_update])</pre>
d) Prediction from a trained model <pre>1/1 ----- 0s 25ms/step inputs: [[0. 0.]], output: 0.0 1/1 ----- 0s 25ms/step inputs: [[0. 1.]], output: 1.0 1/1 ----- 0s 25ms/step inputs: [[1. 0.]], output: 1.0 1/1 ----- 0s 25ms/step inputs: [[1. 1.]], output: 0.0</pre>	<pre>#d.) Prediction from a Trained Model #check the output of the trained model for inputs in X: inputs = tf.expand_dims(inputs, 0) outputs = model.predict(inputs, steps=1) print('inputs: {}, output: {}'.format(inputs, round(outputs.item(), 2))) #e.) Printing the Weights print("Number of layers:", len(model.layers)) print("Weights of the First Layer:\n", model.layers[0].get_weights()) print("Weights of the Second Layer:\n", model.layers[1].get_weights())</pre>
e) Printing the weights of the model <pre>Number of layers: 2 Weights of the first layer: [[[-2.6581159, 2.3568778, -0.6503191], [2.486093, -2.34367, 0.0868491]], dtype=float32], array([-1.6918962, -1.6312462, -0.4888463], dtype=float32)] Weights of the second layer: [[[2.592439,], [2.5727835,], [-0.41802576]], dtype=float32], array([-0.6053341], dtype=float32)]</pre>	



2. Plot of the Malaria dataset

Photo/s of Python output	Photos of Python Codes in Colaboratory
	Laboratory Task #2 <pre>[4] import numpy as np import pickle from matplotlib import pyplot as plt [12] from google.colab import files uploaded = files.upload() Choose Files 2 files • malaria_test.pickle(n/a) - 21491577 bytes, last modified: 6/26/2025 - 100% done • malaria_train.pickle(n/a) - 97823088 bytes, last modified: 6/26/2025 - 100% done Saving malaria_test.pickle to malaria_test (1).pickle Saving malaria_train.pickle to malaria_train (1).pickle [6] data_folder="/content/" [20] #===== #Unpacking Dataset #Get training data with open(data_folder + "malaria_train (1).pickle", 'rb') as handle: get_train_dict = pickle.load(handle) #Get test data with open(data_folder + "malaria_test (1).pickle", 'rb') as handle: get_test_dict = pickle.load(handle) #===== #Sort the Data (Image) and Labels for Training and Testing #Training Dataset----- #Get training data train_data = [] train_label = [] class_size = 2 #Shuffle the indices shuff_idx = np.arange(len(get_train_dict)) for i in range(3): np.random.shuffle(shuff_idx) for i in range(len(get_train_dict)): train_data.append(np.array(get_train_dict[shuff_idx[i]][0]).astype('float32') / 255) train_label.append(np.array(get_train_dict[shuff_idx[i]][1]).astype('float32')) train_data = np.array(train_data) train_label = np.array(train_label) #Testing Dataset----- #Get test data test_data = [] test_label = [] #Shuffle the indices shuff_idx = np.arange(len(get_test_dict)) for i in range(3): np.random.shuffle(shuff_idx) for i in range(len(get_test_dict)): test_data.append(np.array(get_test_dict[shuff_idx[i]][0]).astype('float32') / 255) test_label.append(np.array(get_test_dict[shuff_idx[i]][1]).astype('float32')) test_data = np.array(test_data) test_label = np.array(test_label) #===== #Plot the Data #Function for Plotting----- def plot_graph(images, labels, ax_size=3, figsize=(5, 5), is_gray=False): #Plot image _, ax = plt.subplots(nrows=ax_size, ncols=ax_size, figsize=figsize) idx = 0 for row in ax: for col in row: height, width = images[idx].shape[:2] col.axis('off') img_title = 'Label: ' + str(labels[idx]) col.set_title(img_title) if is_gray: col.imshow(images[idx], cmap='gray') else: col.imshow(images[idx]) idx += 1 #Plotting----- plot_graph(train_data, train_label, ax_size=4, figsize=(7, 7))</pre>



3. Neural Network or Artificial Neural Network (ANN) Model

Photo/s of Python output	Photos of Python Codes in Colaboratory															
a. Create the Model Model: "sequential" <table border="1"> <thead> <tr> <th>Layer (type)</th> <th>Output Shape</th> <th>Param #</th> </tr> </thead> <tbody> <tr> <td>flatten (Flatten)</td> <td>(None, 4096)</td> <td>0</td> </tr> <tr> <td>dense (Dense)</td> <td>(None, 64)</td> <td>262,208</td> </tr> <tr> <td>dense_1 (Dense)</td> <td>(None, 32)</td> <td>2,080</td> </tr> <tr> <td>dense_2 (Dense)</td> <td>(None, 2)</td> <td>66</td> </tr> </tbody> </table> Total params: 264,354 (1.01 MB) Trainable params: 264,354 (1.01 MB) Non-trainable params: 0 (0.00 B)	Layer (type)	Output Shape	Param #	flatten (Flatten)	(None, 4096)	0	dense (Dense)	(None, 64)	262,208	dense_1 (Dense)	(None, 32)	2,080	dense_2 (Dense)	(None, 2)	66	Laboratory Task #3 <pre>[20] import tensorflow as tf import keras from keras.models import Sequential from keras.layers import Dense, Flatten, Conv2D, MaxPooling2D, Dropout from keras.losses import categorical_crossentropy from keras.optimizers import Adam from sklearn.model_selection import RepeatedStratifiedKFold</pre> <pre>[2] #Model Configuration===== #Initial hyperparameters----- batch_size = 50 lr = 1e-3 #Model Configuration----- img_wid, img_hgt, img_chl = 64, 64, 3 class_size = 2 num_epochs = 50 #Define the Neural Network Model----- def neural_net(input_shape, class_size, learning_rate): #Define the model architecture model = Sequential() #input layer model.add(tf.keras.Input(shape=input_shape)) model.add(Flatten()) #hidden layer 1 model.add(Dense(64, activation='relu', kernel_initializer='glorot_uniform')) #hidden layer 2 model.add(Dense(32, activation='relu', kernel_initializer='glorot_uniform')) #output layer model.add(Dense(class_size, activation='softmax', kernel_initializer='glorot_uniform')) #compile the model model.compile(loss=categorical_crossentropy, optimizer=Adam(learning_rate=learning_rate), metrics=['accuracy']) return model #Create the Model neural_model = neural_net(input_shape=(img_wid, img_hgt), class_size=class_size, learning_rate=lr) neural_model.summary()</pre> <pre>[18] #Data Preprocessing #convert the Image Data to Gray Scale (1 Channel instead of 3 Channel RGB) import cv2 gr_data = [] for i in range(len(train_data)): gr_data.append(cv2.cvtColor(train_data[i], cv2.COLOR_RGB2GRAY)) gr_test = [] for i in range(len(test_data)): gr_test.append(cv2.cvtColor(test_data[i], cv2.COLOR_RGB2GRAY)) #plot the converted data plot_graph(gr_data, train_label, ax_size=4, figsize=(7, 7), is_gray=True)</pre> <pre>[19] #train and test the Model #train----- neural_model.fit(np.array(gr_data), np.array(train_label), epochs=num_epochs, batch_size=batch_size, verbose=1) #Test loss, acc = neural_model.evaluate(np.array(gr_test), np.array(test_label), verbose=0) print(i, "Loss:", loss, ", Test Accuracy:", acc)</pre>
Layer (type)	Output Shape	Param #														
flatten (Flatten)	(None, 4096)	0														
dense (Dense)	(None, 64)	262,208														
dense_1 (Dense)	(None, 32)	2,080														
dense_2 (Dense)	(None, 2)	66														
b. Data Preprocessing																
c. Train and Test the Model <pre>Epoch 40/50 200/200 — 1s 5ms/step - accuracy: 0.8093 - loss: 0.4110 Epoch 41/50 200/200 — 1s 5ms/step - accuracy: 0.8193 - loss: 0.3918 Epoch 42/50 200/200 — 1s 6ms/step - accuracy: 0.8204 - loss: 0.3992 Epoch 43/50 200/200 — 2s 9ms/step - accuracy: 0.8245 - loss: 0.3941 Epoch 44/50 200/200 — 2s 5ms/step - accuracy: 0.8260 - loss: 0.3848 Epoch 45/50 200/200 — 1s 5ms/step - accuracy: 0.8279 - loss: 0.3833 Epoch 46/50 200/200 — 1s 6ms/step - accuracy: 0.8302 - loss: 0.3866 Epoch 47/50 200/200 — 1s 5ms/step - accuracy: 0.8348 - loss: 0.3759 Epoch 48/50 200/200 — 1s 5ms/step - accuracy: 0.8315 - loss: 0.3775 Epoch 49/50 200/200 — 1s 5ms/step - accuracy: 0.8337 - loss: 0.3651 Epoch 50/50 200/200 — 1s 5ms/step - accuracy: 0.8386 - loss: 0.3675 1999 Loss: 0.9341700673103333 , Test Accuracy: 0.6054999828338623</pre>																

4. Convolutional Neural Network (CNN) Model

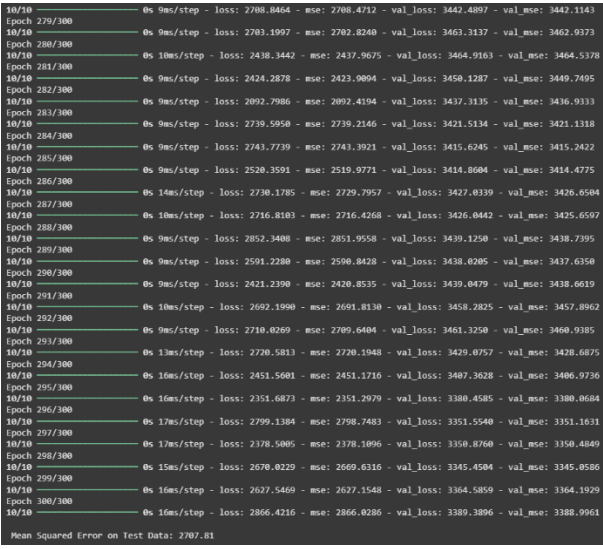
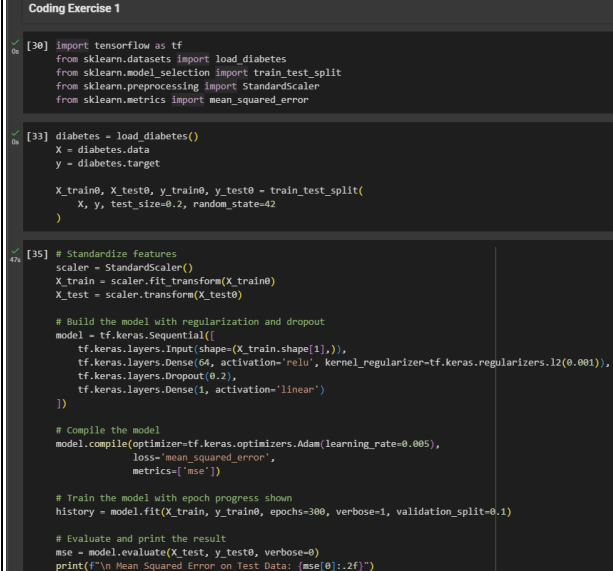
Photo/s of Python output	Photos of Python Codes in Colaboratory																											
a. Create the CNN Model Model: "sequential_1" <table><thead><tr><th>Layer (type)</th><th>Output Shape</th><th>Param #</th></tr></thead><tbody><tr><td>conv2d (Conv2D)</td><td>(None, 60, 60, 16)</td><td>1,216</td></tr><tr><td>max_pooling2d (MaxPooling2D)</td><td>(None, 15, 15, 16)</td><td>0</td></tr><tr><td>conv2d_1 (Conv2D)</td><td>(None, 13, 13, 48)</td><td>6,960</td></tr><tr><td>max_pooling2d_1 (MaxPooling2D)</td><td>(None, 6, 6, 48)</td><td>0</td></tr><tr><td>flatten_1 (Flatten)</td><td>(None, 1728)</td><td>0</td></tr><tr><td>dropout (Dropout)</td><td>(None, 1728)</td><td>0</td></tr><tr><td>dense_3 (Dense)</td><td>(None, 256)</td><td>442,624</td></tr><tr><td>dense_4 (Dense)</td><td>(None, 2)</td><td>514</td></tr></tbody></table> Total params: 451,314 (1.72 MB) Trainable params: 451,314 (1.72 MB) Non-trainable params: 0 (0.00 B)	Layer (type)	Output Shape	Param #	conv2d (Conv2D)	(None, 60, 60, 16)	1,216	max_pooling2d (MaxPooling2D)	(None, 15, 15, 16)	0	conv2d_1 (Conv2D)	(None, 13, 13, 48)	6,960	max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 48)	0	flatten_1 (Flatten)	(None, 1728)	0	dropout (Dropout)	(None, 1728)	0	dense_3 (Dense)	(None, 256)	442,624	dense_4 (Dense)	(None, 2)	514	Laboratory Task #4 <pre>[22] def conv_neural_net(input_shape, class_size, learning_rate): # Define the model architecture model = Sequential() # Input Layer model.add(tf.keras.Input(shape=input_shape)) # Convolutional Layers # Conv Layer 1 with Pooling model.add(Conv2D(16, kernel_size=(5, 5), activation='relu', kernel_initializer='he_normal')) model.add(MaxPooling2D(pool_size=(4, 4))) # Conv Layer 2 with Pooling model.add(Conv2D(48, kernel_size=(3, 3), activation='relu', kernel_initializer='he_normal')) model.add(MaxPooling2D(pool_size=(2, 2))) # Fully Connected Layers model.add(Flatten()) # Convert the layer into 1D matrix model.add(Dropout(0.2)) # Dropout method to prevent overfitting # Fully Connected layer 1 model.add(Dense(256, activation='relu')) # Output Layer model.add(Dense(class_size, activation='softmax')) # Compile the model model.compile(loss=categorical_crossentropy, optimizer=Adam(learning_rate=learning_rate), metrics=['accuracy']) return model</pre> <pre>[23] #Create CNN Model conv_model = conv_neural_net(input_shape=[img_wid, img_hgt, img_chl], class_size=class_size, learning_rate=lr) conv_model.summary() [24] #Train and Test the CNN #Train the neural network using the RGB data conv_model.fit(np.array(train_data), np.array(train_label), epochs=20, batch_size=batch_size, verbose=1) #Test the CNN model (using the same RGB images that it was trained with) loss, acc = conv_model.evaluate(np.array(test_data), np.array(test_label), verbose=0) print(f"Loss: {loss}, Test Accuracy: {acc}")</pre>
Layer (type)	Output Shape	Param #																										
conv2d (Conv2D)	(None, 60, 60, 16)	1,216																										
max_pooling2d (MaxPooling2D)	(None, 15, 15, 16)	0																										
conv2d_1 (Conv2D)	(None, 13, 13, 48)	6,960																										
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 48)	0																										
flatten_1 (Flatten)	(None, 1728)	0																										
dropout (Dropout)	(None, 1728)	0																										
dense_3 (Dense)	(None, 256)	442,624																										
dense_4 (Dense)	(None, 2)	514																										
b. Test and Train the CNN Epoch 1/20 200/200 — 26s 119ms/step - accuracy: 0.5974 - loss: 0.8241 Epoch 2/20 200/200 — 23s 116ms/step - accuracy: 0.7414 - loss: 0.5181 Epoch 3/20 200/200 — 41s 117ms/step - accuracy: 0.8105 - loss: 0.4181 Epoch 4/20 200/200 — 42s 120ms/step - accuracy: 0.8724 - loss: 0.3053 Epoch 5/20 200/200 — 39s 111ms/step - accuracy: 0.9162 - loss: 0.2147 Epoch 6/20 200/200 — 42s 117ms/step - accuracy: 0.9471 - loss: 0.1493 Epoch 7/20 200/200 — 41s 117ms/step - accuracy: 0.9561 - loss: 0.1267 Epoch 8/20 200/200 — 40s 112ms/step - accuracy: 0.9596 - loss: 0.1138 Epoch 9/20 200/200 — 42s 117ms/step - accuracy: 0.9687 - loss: 0.0905 Epoch 10/20 200/200 — 41s 117ms/step - accuracy: 0.9727 - loss: 0.0816 Epoch 11/20 200/200 — 23s 117ms/step - accuracy: 0.9756 - loss: 0.0671 Epoch 12/20 200/200 — 23s 117ms/step - accuracy: 0.9779 - loss: 0.0650 Epoch 13/20 200/200 — 41s 117ms/step - accuracy: 0.9846 - loss: 0.0476 Epoch 14/20 200/200 — 40s 111ms/step - accuracy: 0.9831 - loss: 0.0483 Epoch 15/20 200/200 — 42s 117ms/step - accuracy: 0.9851 - loss: 0.0457 Epoch 16/20 200/200 — 41s 116ms/step - accuracy: 0.9856 - loss: 0.0423 Epoch 17/20 200/200 — 40s 112ms/step - accuracy: 0.9878 - loss: 0.0359 Epoch 18/20 200/200 — 42s 117ms/step - accuracy: 0.9874 - loss: 0.0339 Epoch 19/20 200/200 — 41s 116ms/step - accuracy: 0.9907 - loss: 0.0277 Epoch 20/20 200/200 — 40s 113ms/step - accuracy: 0.9913 - loss: 0.0265 1999 Loss: 0.3229798376560211 , Test Accuracy: 0.925000011920929																												



B. Problem Exercises:

(Please include the following: 1. Photos of the codes and 2. Outputs after executing the Python code.)

Problem 1:

Photo/s of Python output	Photos of Python Codes in Colaboratory
 <pre> Epoch 279/300 0s 9ms/step - loss: 2708.8464 - mse: 2708.4712 - val_loss: 3442.4897 - val_mse: 3442.1143 Epoch 280/300 0s 9ms/step - loss: 2703.1997 - mse: 2702.8240 - val_loss: 3463.3137 - val_mse: 3462.9373 Epoch 281/300 0s 10ms/step - loss: 2418.3442 - mse: 2437.9675 - val_loss: 3464.9163 - val_mse: 3464.5378 Epoch 282/300 0s 9ms/step - loss: 2424.2878 - mse: 2423.9094 - val_loss: 3450.1287 - val_mse: 3449.7495 Epoch 283/300 0s 9ms/step - loss: 2092.7986 - mse: 2092.4194 - val_loss: 3437.3135 - val_mse: 3436.9333 Epoch 284/300 0s 9ms/step - loss: 2739.5950 - mse: 2739.2146 - val_loss: 3421.5134 - val_mse: 3421.1318 Epoch 285/300 0s 9ms/step - loss: 2743.7739 - mse: 2743.3921 - val_loss: 3415.6245 - val_mse: 3415.2422 Epoch 286/300 0s 9ms/step - loss: 2520.3591 - mse: 2519.9771 - val_loss: 3414.8604 - val_mse: 3414.4775 Epoch 287/300 0s 14ms/step - loss: 2730.1785 - mse: 2729.7957 - val_loss: 3427.0339 - val_mse: 3426.0504 Epoch 288/300 0s 10ms/step - loss: 2716.8103 - mse: 2716.4268 - val_loss: 3426.0442 - val_mse: 3425.6597 Epoch 289/300 0s 9ms/step - loss: 2852.3488 - mse: 2851.9558 - val_loss: 3439.1250 - val_mse: 3438.7395 Epoch 290/300 0s 9ms/step - loss: 2591.2280 - mse: 2590.8428 - val_loss: 3438.0205 - val_mse: 3437.6350 Epoch 291/300 0s 10ms/step - loss: 2421.2390 - mse: 2420.8535 - val_loss: 3439.0479 - val_mse: 3438.6619 Epoch 292/300 0s 10ms/step - loss: 2692.1990 - mse: 2691.8130 - val_loss: 3458.2825 - val_mse: 3457.8962 Epoch 293/300 0s 9ms/step - loss: 2710.8269 - mse: 2709.6404 - val_loss: 3461.3250 - val_mse: 3460.9385 Epoch 294/300 0s 13ms/step - loss: 2720.5813 - mse: 2720.1948 - val_loss: 3429.0757 - val_mse: 3428.6875 Epoch 295/300 0s 10ms/step - loss: 2451.5601 - mse: 2451.1716 - val_loss: 3407.3628 - val_mse: 3406.9736 Epoch 296/300 0s 10ms/step - loss: 2351.6873 - mse: 2351.2979 - val_loss: 3308.4585 - val_mse: 3308.0684 Epoch 297/300 0s 17ms/step - loss: 2799.1384 - mse: 2798.7483 - val_loss: 3351.5540 - val_mse: 3351.1631 Epoch 298/300 0s 15ms/step - loss: 2378.5005 - mse: 2378.1096 - val_loss: 3350.8760 - val_mse: 3350.4849 Epoch 299/300 0s 10ms/step - loss: 2670.8229 - mse: 2669.6316 - val_loss: 3345.4504 - val_mse: 3345.0586 Epoch 300/300 0s 10ms/step - loss: 2627.5469 - mse: 2627.1548 - val_loss: 3364.5859 - val_mse: 3364.1929 Epoch 300/300 0s 10ms/step - loss: 2866.4216 - mse: 2866.0286 - val_loss: 3389.3896 - val_mse: 3388.9961 Mean Squared Error on Test Data: 2707.81 </pre>	 <pre> Coding Exercise 1 [38] import tensorflow as tf from sklearn.datasets import load_diabetes from sklearn.model_selection import train_test_split from sklearn.preprocessing import StandardScaler from sklearn.metrics import mean_squared_error [33] diabetes = load_diabetes() X = diabetes.data y = diabetes.target X_train0, X_test0, y_train0, y_test0 = train_test_split(X, y, test_size=0.2, random_state=42) [35] # Standardize features scaler = StandardScaler() X_train = scaler.fit_transform(X_train0) X_test = scaler.transform(X_test0) # Build the model with regularization and dropout model = tf.keras.Sequential([tf.keras.layers.Input(shape=(X_train.shape[1])), tf.keras.layers.Dense(64, activation='relu', kernel_regularizer=tf.keras.regularizers.l2(0.001)), tf.keras.layers.Dropout(0.2), tf.keras.layers.Dense(1, activation='linear')]) # Compile the model model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.005), loss='mean_squared_error', metrics=['mse']) # Train the model with epoch progress shown history = model.fit(X_train, y_train0, epochs=300, verbose=1, validation_split=0.1) # Evaluate and print the result mse = model.evaluate(X_test, y_test0, verbose=0) print(f"Mean Squared Error on Test Data: {mse[0]:.2f}") </pre>



Problem 2:

Photo/s of Python output	Photos of Python Codes in Colaboratory
<pre> Final y_train shape: (60000, 10) Final y_test shape: (10000, 10) Epoch 1/15 211/211 --- 23s 48ms/step - accuracy: 0.6989 - loss: 0.9451 - val_accuracy: 0.1898 - val_loss: 8.6527 Epoch 2/15 211/211 --- 4s 17ms/step - accuracy: 0.8477 - loss: 0.4102 - val_accuracy: 0.3672 - val_loss: 3.9567 Epoch 3/15 211/211 --- 3s 15ms/step - accuracy: 0.8729 - loss: 0.3439 - val_accuracy: 0.6710 - val_loss: 0.3753 Epoch 4/15 211/211 --- 5s 16ms/step - accuracy: 0.8885 - loss: 0.3023 - val_accuracy: 0.8983 - val_loss: 0.2716 Epoch 5/15 211/211 --- 4s 17ms/step - accuracy: 0.8961 - loss: 0.2801 - val_accuracy: 0.9127 - val_loss: 0.2415 Epoch 6/15 211/211 --- 3s 15ms/step - accuracy: 0.9059 - loss: 0.2595 - val_accuracy: 0.9127 - val_loss: 0.2350 Epoch 7/15 211/211 --- 5s 15ms/step - accuracy: 0.9125 - loss: 0.2387 - val_accuracy: 0.9190 - val_loss: 0.2265 Epoch 8/15 211/211 --- 3s 16ms/step - accuracy: 0.9142 - loss: 0.2279 - val_accuracy: 0.9178 - val_loss: 0.2273 Epoch 9/15 211/211 --- 3s 15ms/step - accuracy: 0.9208 - loss: 0.2112 - val_accuracy: 0.9177 - val_loss: 0.2360 Epoch 10/15 211/211 --- 3s 15ms/step - accuracy: 0.9267 - loss: 0.1986 - val_accuracy: 0.9245 - val_loss: 0.2101 Epoch 11/15 211/211 --- 3s 15ms/step - accuracy: 0.9276 - loss: 0.1928 - val_accuracy: 0.9285 - val_loss: 0.1935 Epoch 12/15 211/211 --- 3s 16ms/step - accuracy: 0.9328 - loss: 0.1824 - val_accuracy: 0.9185 - val_loss: 0.2240 Epoch 13/15 211/211 --- 3s 15ms/step - accuracy: 0.9341 - loss: 0.1743 - val_accuracy: 0.9268 - val_loss: 0.2168 Epoch 14/15 211/211 --- 5s 16ms/step - accuracy: 0.9383 - loss: 0.1662 - val_accuracy: 0.9342 - val_loss: 0.1919 Epoch 15/15 211/211 --- 5s 16ms/step - accuracy: 0.9411 - loss: 0.1568 - val_accuracy: 0.9313 - val_loss: 0.1999 Test accuracy: 92.46% </pre>	<pre> Coding Exercise 2 [2] import numpy as np import tensorflow as tf from keras.models import Sequential from keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout, BatchNormalization from keras.utils import to_categorical from keras.datasets import fashion_mnist from keras.optimizers import Adam from keras.callbacks import EarlyStopping [3] (x_train, y_train), (x_test, y_test) = fashion_mnist.load_data() print('x_train shape:', x_train.shape) print(x_train.shape[0], 'train samples') print(x_test.shape[0], 'test samples') [4] # Reshape and normalize x_train = x_train.reshape(-1, 28, 28, 1).astype('float32') / 255.0 x_test = x_test.reshape(-1, 28, 28, 1).astype('float32') / 255.0 # One-hot encode the labels if y_train.ndim == 3 and y_train.shape[1:] == (10, 10): y_train = np.array([np.diag(y) for y in y_train]) # fix double one-hot if y_train.ndim == 1: y_train = to_categorical(y_train, 10) y_test = to_categorical(y_test, 10) # Build the CNN model model = Sequential([Conv2D(32, kernel_size=3, activation='relu', input_shape=(28, 28, 1)), BatchNormalization(), Conv2D(64, kernel_size=3, activation='relu'), MaxPooling2D(pool_size=2), Dropout(0.25), Conv2D(128, kernel_size=3, activation='relu'), BatchNormalization(), MaxPooling2D(pool_size=2), Dropout(0.25), Flatten(), Dense(128, activation='relu'), Dropout(0.3), Dense(10, activation='softmax') # MUST be softmax for multiclass classification]) # Compile the model with a reasonable learning rate model.compile(optimizer=Adam(learning_rate=0.0005), # Not too high loss='categorical_crossentropy', # MUST be categorical_crossentropy for one-hot metrics=['accuracy']) # Early stopping to prevent overfitting early_stop = EarlyStopping(monitor='val_accuracy', patience=3, restore_best_weights=True) print("Final y_train shape:", y_train.shape) print("Final y_test shape:", y_test.shape) # Train the model history = model.fit(x_train, y_train, batch_size=256, epochs=15, validation_split=0.1, callbacks=[early_stop], verbose=1) # Evaluate test_loss, test_acc = model.evaluate(x_test, y_test, verbose=0) print(f"Test accuracy: {test_acc * 100:.2f}%") </pre>



C. Findings ,Observations, and Comments:

Guiding Questions :

1. *What did you learn in the guided exercises? Did you learn something new?*
2. *What are your findings, observations, and comments in the guided exercises?*
3. *What are your findings, observations, and comments in the problem exercises?*
4. *Did you experience any difficulties in solving the problem exercises?*

Throughout the guided exercises, I learned how to use TensorFlow and Keras to build simple and deep neural networks. I also learned about the difference between the outputs of trained and untrained models after and before training and I discovered how training loss decreases as the model learns from the dataset. Lastly, I learned how to preprocess image data, like grayscale conversion, and to prepare datasets for classification tasks. I have observed that Neural networks were able to approximate the XOR logic gate after training, which proves the concept of learning from patterns. The malaria dataset also demonstrated how real-world data is used in binary classification and I also observed that Convolution layers were effective in learning complex and abstract features from image data and the CNN models showed significant improvement in detecting infected versus uninfected cells. I had trouble in installing the TensorFlow and Keras in my laptop and on how I will make use of it in Google Colaboratory since it was my first time using those libraries and software applications. Thankfully, I managed to figure it out with the help of the resources online like YouTube. I watched some tutorials on how to set up those materials and it really helped me understand better the laboratory tasks. I also found it amazing because there were some outputs that were images, and I didn't expect that it would be possible. All in all, I think this laboratory task will serve as my foundation to CNN and to other deep learning models that I may encounter again in the future.